TOURS APPLICATION

Simple Explanation of BackgroundService Cleanup and Quartz ETL

Exam-ready project explanation with architecture, flow, testing, and final submission fixes

Core idea: The application runs two automatic jobs: one cleans old cancelled bookings from the local database, and one synchronizes tours/offers from an external SQL Server database into the local SQLite database.

Prepared as a clean PDF study document

Table of Contents

- Part 1: The Big Picture
- Part 2: Why The Project Has Four Layers
- Part 3: JOB 1 - Booking Cleanup Explained Step By Step
- Part 4: JOB 2 - Quartz ETL Explained Step By Step
- Part 5: What Program.Cs Actually Does
- Part 6: How To Test And Know It Works
- Part 7: Simple Presentation Answer
- Part 8: Two Changes Required Before Final Submission

This document explains the project as if we are following the application step by step. Do not try to memorize every code line. First understand who is responsible for each action and how the files communicate.

Part 1: The Big Picture

Our application has its own local database called app.db.

The local database contains:

- Bookings
- Tours
- Offers
- TravelAgencies
- Other application data

We implemented two automatic jobs:

JOB 1 - BOOKING CLEANUP

Every three minutes, the application checks its local database.

It finds bookings that:

- Have Status = Cancelled
- Were created more than seven days ago

It then deletes those bookings.

JOB 2 - TOUR ETL

Every five minutes, the application connects to another company's external SQL Server database.

It reads:

- ToursDirectory
- TourOfferings

It converts that external data into our application's Tour and Offers objects.

It then saves or updates those objects in our local app.db database.

The easiest way to remember the difference is:

- BackgroundService cleans old data already inside our application.
- Quartz imports and synchronizes data from an external database.

Part 2: Why The Project Has Four Layers

Imagine the application as a company with four departments.

1. ToursApplication.Domain - "What does our data look like?"

The Domain project contains definitions of the application's data.

For example:

- Booking.cs says what information one booking has.
- Tour.cs says what information one tour has.
- Offers.cs says what information one offer has.
- BookingStatus.cs defines Pending, Paid, and Cancelled.

The Domain project does not connect to the database.

It only defines the shapes and meanings of the data.

Example:

Booking has:

- Id
- Status
- DateCreated
- TourId
- TravelAgencyId

The cleanup job needs Status and DateCreated to decide whether a booking must

be deleted.

ExternalModels is also inside Domain.

These models describe data from the external database:

- LegacyToursDirectory.cs describes one ToursDirectory row.
- LegacyTourOfferings.cs describes one TourOfferings row.

They are called legacy models because they represent the other database, not

our local database.

2. ToursApplication.Repository - "How do we communicate with databases?"

The Repository project performs database work.

It contains two database contexts because we use two databases:

- ApplicationDbContext connects to our local app.db SQLite database.
- LegacyTourDbContext connects to the external ISLegacyDb SQL Server database.

It also contains repositories.

A repository is a class whose job is to read, insert, update, or delete

database data.

Important repositories:

- Repository<Booking> works with local bookings.
- LegacyTourRepository reads external tours and offerings.
- TourRepository saves Tours and Offers into our local database.

3. ToursApplication.Service - "What work should the application perform?"

The Service project contains application actions and scheduled jobs.

Important files:

- BookingService.cs contains booking-related actions.

- BookCleanUpBackgroundService.cs controls the automatic booking cleanup.
- QuartzTourEtlJob.cs controls one ETL synchronization execution.

The jobs do not know SQL. They ask services or repositories to do database work.

4. ToursApplication.Web - "How does everything start and connect?"

The Web project is the application entry point.

Program.cs starts the application and connects all classes using dependency injection.

appsettings.json stores database connection strings.

Controllers provide HTTP endpoints that can be tested using Postman.

For example:

- GET /api/Booking
- GET /api/Tour
- GET /api/Offers

The controllers do not start the automatic jobs. The jobs start by themselves when the application starts.

Part 3: JOB 1 - Booking Cleanup Explained Step By Step

The purpose of this job is simple:

"Every three minutes, remove cancelled bookings older than seven days."

STEP 1: The Booking model provides the required information

File:

ToursApplication.Domain/Models/Booking.cs

Booking contains:

- Status
- DateCreated, inherited from BaseAuditableEntity

The job checks whether:

Status == Cancelled

and whether:

DateCreated is older than seven days

STEP 2: IBookingService announces available booking actions

File:

ToursApplication.Service/Interface/IBookingService.cs

An interface is like a menu of available actions.

It contains:

GetOldCancelledBookingsAsync(DateTime cutoffDate)

This means:

"Give me all cancelled bookings created before this date."

It also contains:

DeleteByIdAsync(Guid id)

This means:

"Delete the booking with this ID."

STEP 3: BookingService performs the booking logic

File:

ToursApplication.Service/Implementation/BookingService.cs

BookingService implements IBookingService.

GetOldCancelledBookingsAsync filters local bookings:

- Status must be Cancelled.
- DateCreated must be before the cutoff date.

BookingService does not write SQL itself. It asks IRepository<Booking> to query the database.

DeleteByIdAsync:

1. Finds the booking.

2. Sends it to Repository<Booking>.

3. Repository<Booking> deletes it from ApplicationDbContext.

4. ApplicationDbContext saves the change into local app.db.

STEP 4: BookCleanUpBackgroundService decides when cleanup happens

File:

ToursApplication.Service/Jobs/BookCleanUpBackgroundService.cs

This class inherits from BackgroundService.

BackgroundService is an ASP.NET Core class for work that runs automatically while the application is open.

Its important method is:

ExecuteAsync(CancellationToken stoppingToken)

ASP.NET Core automatically calls ExecuteAsync when the application starts.

Why is there a while loop?

The loop keeps the job alive:

while (!stoppingToken.IsCancellationRequested)

In simple words:

"Keep repeating this work until the application is shutting down."

What is CancellationToken?

stoppingToken tells the job when the application is stopping.

Without it, the job may continue trying to work while the application is

shutting down.

Why create a scope?

The background service lives for the entire application lifetime.

BookingService and ApplicationDbContext should be created for a shorter scope.

The job creates a new scope and gets IBookingService:

```
using var scope = _serviceScopeFactory.CreateScope();
var bookingService =
scope.ServiceProvider.GetRequiredService<IBookingService>();
```

In simple words:

"Create a fresh working environment and give me a BookingService for this

cleanup execution."

What happens during one cleanup execution?

1. The job logs that cleanup started.

2. It asks BookingService for old cancelled bookings.

3. It loops through the returned bookings.

4. It asks BookingService to delete each booking.

5. It logs success or errors.

6. It waits three minutes.

7. The loop starts again.

Why Task.Delay?

```
await Task.Delay(TimeSpan.FromMinutes(3), stoppingToken);
```

Task.Delay pauses the job for three minutes without freezing the application.

During this delay:

- Controllers can still receive requests.
- Quartz can still run.
- Other application work can continue.

IMPORTANT FIX FOR JOB 1

The current code uses:

```
DateTime.UtcNow.AddMinutes(-7)
```

That means seven minutes ago.

The assignment requires seven days, so it must be:

```
DateTime.UtcNow.AddDays(-7)
```

FULL JOB 1 CONNECTION

BookCleanUpBackgroundService

asks IBookingService

IBookingService

is implemented by BookingService

BookingService

asks IRepository<Booking>

Repository<Booking>

uses ApplicationDbContext

ApplicationDbContext

deletes data from the local Bookings table in app.db

JOB 1 FLOW IN ONE SENTENCE

Application starts -> BackgroundService starts -> BookingService finds old cancelled bookings -> Repository deletes them from app.db -> job waits three minutes -> process repeats.

Part 4: JOB 2 - Quartz ETL Explained Step By Step

The purpose of this job is:

"Every five minutes, copy and synchronize tour information from the external SQL Server database into our local app.db database."

What does ETL mean?

ETL means Extract, Transform, Load.

Extract:

Read data from the external database.

Transform:

Convert external data into the format our application understands.

Load:

Save or update the converted data in our local database.

What is Quartz?

Quartz is a scheduling library.

BackgroundService uses a loop and Task.Delay.

Quartz uses jobs and triggers.

- Quartz job says what must happen.
- Quartz trigger says when it must happen.
- Quartz scheduler activates the job at the correct time.

The assignment specifically requires Quartz for the ETL task.

STEP 1: The external connection string identifies the external database

File:

ToursApplication.Web/appsettings.json

LegacyConnection contains:

- Server address
- Port
- Username
- Password
- Database name

This information allows the application to connect to ISLegacyDb.

It is not used by Postman.

It is used by Entity Framework Core to connect directly to SQL Server.

STEP 2: LegacyTourDbContext represents the external database

File:

ToursApplication.Repository/LegacyTourDbContext.cs

LegacyTourDbContext contains:

ToursDirectory

TourOfferings

These DbSet properties represent external SQL Server tables.

It maps:

- ToursDirectory.Name as the primary key.
- AgencyName + TourName as the TourOfferings composite primary key.

LegacyTourDbContext only reads external data.

It does not save local Tours or Offers.

STEP 3: External models represent external rows

Files:

ToursApplication.Domain/ExternalModels/LegacyToursDirectory.cs

ToursApplication.Domain/ExternalModels/LegacyTourOfferings.cs

LegacyToursDirectory represents:

Name

Capacity

LegacyTourOfferings represents:

AgencyName

TourName

These classes match the external table columns.

STEP 4: LegacyTourRepository extracts and transforms data

File:

ToursApplication.Repository/Implementation/LegacyTourRepository.cs

This repository reads external records using LegacyTourDbContext.

GetTourAsync

GetTourAsync reads every row from external ToursDirectory.

It converts each external row into a local Tour:

external Name -> local Tour.Name

external Capacity -> local Tour.Capacity

It must also create a local Tour.Id.

The external Tour does not have a Guid ID. Its key is Name.

Our local Tour requires a Guid ID.

That is why GuidHelper is used.

STEP 5: GuidHelper creates stable IDs

File:

ToursApplication.Domain/ExternalModels/GuidHelper.cs

GuidHelper receives:

- Entity type, for example "Tour"
- External key, for example "Summer Adventure"

It returns a Guid.

The important rule is:

Same input -> same Guid every time

Example:

GuidHelper.FromLegacyId("Tour", "Summer Adventure")

always returns the same Tour ID.

Why is this important?

If Guid.NewGuid() were used:

- First synchronization creates one ID.
- Second synchronization creates a different ID.
- The same Tour would be inserted again as a duplicate.

With GuidHelper:

- First synchronization inserts the Tour.
- Next synchronization finds the same ID and updates the Tour.

GetOffersAsync

GetOffersAsync reads every external TourOfferings row.

An external offering contains names:

AgencyName

TourName

But our local Offers entity requires IDs:

TravelAgencyId

TourId

Therefore, LegacyTourRepository performs two mappings.

Mapping TourName to TourId

It uses the same GuidHelper rule:

GuidHelper.FromLegacyId("Tour", TourName)

Because the Tour was created using the same rule, the Offer receives the correct TourId.

Mapping AgencyName to TravelAgencyId

TravelAgencies already exist in our local database.

LegacyTourRepository reads local TravelAgencies from ApplicationDbContext.

It creates a dictionary:

Agency name -> Agency ID

Example:

"Sunny Escapes" -> D63F8834-...

When the external offering has AgencyName = "Sunny Escapes", the repository uses the existing local Sunny Escapes ID.

If an external AgencyName does not exist locally, that offering is skipped.

Creating the Offer ID

The external offering key contains both AgencyName and TourName.

The local Offer ID is created using both:

GuidHelper.FromLegacyId(

"Offer",

AgencyName + ":" + TourName)

The same external offering therefore receives the same local ID every time.

STEP 6: TourRepository loads data into our local database

File:

ToursApplication.Repository/Implementation/TourRepository.cs

LegacyTourRepository reads and transforms data.

TourRepository saves that transformed data.

TourRepository uses ApplicationDbContext because it writes into local app.db.

It has two methods:

BulkInsertOrUpdateToursAsync

BulkInsertOrUpdateOffersAsync

BulkInsertOrUpdate means:

- If the ID does not exist, insert the record.
- If the ID already exists, update the record.

The job does not manually check every record with an if statement.

BulkInsertOrUpdateAsync performs the insert-or-update operation using the

stable IDs created by GuidHelper.

STEP 7: QuartzTourEtlJob coordinates one synchronization

File:

ToursApplication.Service/Jobs/QuartzTourEtlJob.cs

This class implements IJob, which means Quartz can execute it.

During one execution:

1. It asks ILegacyTourRepository for transformed Tours.

2. It asks ITourRepository to insert or update local Tours.

3. It asks ILegacyTourRepository for transformed Offers.

4. It asks ITourRepository to insert or update local Offers.

5. It logs how many Tours and Offers were synchronized.

Why save Tours before Offers?

Offers contain TourId.

The referenced Tour must exist before the Offer can be saved.

QuartzTourEtlJob has no loop and no Task.Delay.

Quartz decides when to call it.

STEP 8: Program.cs schedules Quartz

Program.cs creates a Quartz job identity:

"tour-etl"

It connects QuartzTourEtlJob to a trigger.

The required schedule is:

0 0/5 * * * ?

This means:

"At second zero, run every five minutes."

Program.cs also starts Quartz with the application:

builder.Services.AddQuartzHostedService(...)

IMPORTANT FIX FOR JOB 2

The current code uses:

0 0/1 * * * ?

That runs every one minute and is useful for testing.

The assignment requires every five minutes, so before submission use:

```
0 0/5 * * * ?
```

FULL JOB 2 CONNECTION

Quartz scheduler

triggers QuartzTourEtlJob

QuartzTourEtlJob

asks ILegacyTourRepository for data

ILegacyTourRepository

is implemented by LegacyTourRepository

LegacyTourRepository

reads external data using LegacyTourDbContext
and reads local agencies using ApplicationDbContext

LegacyTourRepository

returns transformed Tour and Offers objects

QuartzTourEtlJob

sends those objects to ITourRepository

ITourRepository

is implemented by TourRepository

TourRepository

saves or updates Tours and Offers using ApplicationDbContext

ApplicationDbContext

writes data into local app.db

JOB 2 FLOW IN ONE SENTENCE

Application starts -> Quartz starts -> every five minutes Quartz runs the ETL job -> external ToursDirectory and TourOfferings are read -> records are converted into Tour and Offers -> local app.db is inserted or updated -> job finishes until the next Quartz trigger.

Part 5: What Program.Cs Actually Does

Program.cs is where we tell the application which concrete class should be used when another class asks for an interface.

This is called dependency injection.

Local database registration

```
AddDbContext<ApplicationDbContext>(...UseSqlite...)
```

Meaning:

"ApplicationDbContext connects to our local SQLite app.db database."

External database registration

```
AddDbContext<LegacyTourDbContext>(...UseSqlServer...)
```

Meaning:

"LegacyTourDbContext connects to the external SQL Server database using LegacyConnection."

Generic repository registration

```
AddScoped(typeof(IRepository<>), typeof(Repository<>))
```

Meaning:

"When a class asks for IRepository<Booking>, give it Repository<Booking>."

Booking service registration

```
AddTransient<IBookingService, BookingService>()
```

Meaning:

"When a class asks for IBookingService, give it BookingService."

ETL repository registrations

```
AddScoped<ILegacyTourRepository, LegacyTourRepository>()
AddScoped<ITourRepository, TourRepository>()
```

Meaning:

"When QuartzTourEtlJob asks for these interfaces, create these repository implementations."

BackgroundService registration

```
AddHostedService<BookCleanUpBackgroundService>()
```

Meaning:

"Start the booking cleanup job automatically when the application starts."

Quartz registration

```
AddQuartz(...)
AddQuartzHostedService(...)
```

Meaning:

"Register the ETL job, create its schedule, and start Quartz automatically with the application."

Part 6: How To Test And Know It Works

The jobs are automatic. Postman does not directly start them.

Start the application and watch the console.

For booking cleanup, look for:

Booking cleanup job started...
Fetched X old cancelled bookings
Booking cleanup job finished successfully.

This proves the BackgroundService executed.

For Quartz ETL, look for:

Tour ETL job started...
Tour ETL job finished. Tours: X, Offers: Y

This proves Quartz executed the ETL job.

Use Postman after the jobs run:

GET http://localhost:5253/api/Booking
GET http://localhost:5253/api/Tour
GET http://localhost:5253/api/Offers

Postman shows the current records in the local application database.
Run the ETL job multiple times. The same external records should not be duplicated because GuidHelper creates the same IDs every time.

Part 7: Simple Presentation Answer

You can explain the project like this:

"I implemented two automatic jobs.
The first job uses BackgroundService. It starts with the application and runs in a loop every three minutes. It asks BookingService for cancelled bookings older than seven days. BookingService uses the repository and local ApplicationDbContext to delete those bookings from app.db.
The second job uses Quartz because the task requires a Quartz schedule. Every five minutes, Quartz runs QuartzTourEtlJob. The job uses LegacyTourRepository to read ToursDirectory and TourOfferings from the external SQL Server database. The repository transforms external names into local Tour and Offers objects. GuidHelper creates stable Guid IDs so records are updated instead of duplicated. TourRepository then inserts or updates the local SQLite database.
Program.cs connects all interfaces, implementations, database contexts, and scheduled jobs through dependency injection. Because of those registrations, both jobs start automatically when the application runs."

Part 8: Two Changes Required Before Final Submission

1. In BookCleanUpBackgroundService, use:

DateTime.UtcNow.AddDays(-7)

Do not leave:

DateTime.UtcNow.AddMinutes(-7)

2. In Program.cs, use the required five-minute Quartz schedule:

.WithCronSchedule("0 0/5 * * * ?")

Do not leave the one-minute testing schedule:

.WithCronSchedule("0 0/1 * * * ?")

Final Submission Checklist

Requirement	Correct final value
Booking cleanup cutoff	DateTime.UtcNow.AddDays(-7)
Quartz ETL schedule	.WithCronSchedule("0 0/5 * * * ?")